Linksammlung v2.6 - Technische Dokumentation

Inhaltsverzeichnis

1. System-Übersicht
2. Architektur
3. Dateistruktur
4. Datenmodell
5. Kernfunktionen
6. API-Referenz
7. Konfiguration
8. Deployment
9. Performance
10. Sicherheit
11. Browser-Kompatibilität
12. Wartung

System-Übersicht

Technische Spezifikationen

- **Frontend**: Vanilla JavaScript (ES6+), HTML5, CSS3
- **Styling**: CSS Grid, Flexbox, CSS Animations
- **Persistierung**: localStorage (Browser)
- **Architektur**: Client-side SPA, modulare Struktur
- **Build-System**: Keine Build-Tools erforderlich
- **Dependencies**: Keine externen Bibliotheken

Systemanforderungen

Browser-Support:

- Chrome 80+
- Firefox 75+
- Safari 13+
- Edge 80+

Features:

- JavaScript enabled
- localStorage support (mindestens 5MB)
- CSS3 support
- Drag & Drop API support

Architektur

Komponentenarchitektur

```
Linksammlung Application
├── Core System
│   ├── Config Management
│   ├── Data Persistence
│   └── Event Handling
├── UI Components
│   ├── Category Manager
│   ├── Link Manager
│   ├── Search System
│   └── Modal System
├── Feature Modules
│   ├── Favorites System
│   ├── Drag & Drop
│   ├── Import/Export
│   └── Emoji Picker
└── Utility Systems
    ├── Welcome Loader
    ├── Message System
    └── Context Menus
```

Datenfluss

```
User Action → Event Handler → Data Model Update → localStorage → UI Render → User Feedback
```

Modulare Struktur

- **script.js**: Hauptlogik und Orchestrierung
- **default-config.js**: Standard-Konfigurationsdaten
- **emoji-database.js**: Emoji-Kategorien und -Daten
- **welcome-config-loader.js**: Onboarding-System
- **styles.css**: Styling und Animationen

Dateistruktur

```
/
├── Linksammlung.html ..... # Haupt-HTML-Datei
├── CSS/
│   └── styles.css          # Styling (strukturiert nach BEM-ähnlicher Methodik)
├── JS/
│   ├── script.js ........... # Hauptlogik (16 Sektionen)
│   ├── emoji-database.js    # Emoji-Kategorien-System
│   └── welcome-config-loader.js # Onboarding-System
└── Config/
    └── default-config.js ... # Standard-Konfiguration
```

Datei-Abhängigkeiten

```
Linksammlung.html
├── CSS/styles.css
├── Config/default-config.js (optional)
├── JS/emoji-database.js
├── JS/script.js
└── JS/welcome-config-loader.js
```

Datenmodell

Hauptkonfiguration

typescript

```typescript
interface Config {
  categories: Record<string, Category>;
  categoryOrder: string[];
}

interface Category {
  title: string;
  icon: string;
  collapsed: boolean;
  links: Link[];
}

interface Link {
  title: string;
  url: string;
  description: string;
  icon: string;
  favorite: boolean;
}
```

localStorage-Schema

Schlüssel: `linkConfig`

Struktur:

```
json
```

```
{
  "categories": {
    "category_key": {
      "title": "Category Name",
      "icon": "📁",
      "collapsed": false,
      "links": [
        {
          "title": "Link Title",
          "url": "https://example.com",
          "description": "Description",
          "icon": "🔗",
          "favorite": false
        }
      ]
    }
  },
  "categoryOrder": ["category_key"]
}
```

Emoji-Datenbank

```typescript
interface EmojiDatabase {
  [categoryKey: string]: EmojiCategory;
}

interface EmojiCategory {
  name: string;
  icon: string;
  emojis: EmojiData[];
}

interface EmojiData {
  emoji: string;
  name: string;
}
```

Kernfunktionen

1. Konfigurationsmanagement

Funktionen:

- `loadConfig()`: Lädt Konfiguration mit Prioritätslogik

- `saveConfig()`: Speichert Konfiguration in localStorage
- `validateConfig()`: Validiert und repariert Konfigurationsdaten
- `resetToDefault()`: Setzt auf Standard zurück

Prioritätsreihenfolge:

1. localStorage (existierende Konfiguration)
2. default-config.js (Standard-Konfiguration)
3. Fallback-Konfiguration (hardcoded)

2. Favoriten-System

Implementierung:

- Link-Objekte erweitert um `favorite: boolean`
- Seitenleiste mit Favoriten-Übersicht
- Counter-System mit Echtzeit-Updates
- Favoriten-Toggle über Kontextmenü

API:

```javascript
// Favoriten-Status ändern
toggleLinkFavorite()

// Favoriten-Liste aktualisieren
updateFavoritesList()

// Favoriten-Anzahl aktualisieren
updateFavoritesCount()
```

3. Suche

Funktionen:

- Volltext-Suche über Titel, Beschreibung, URL
- Relevanz-Scoring-System
- Keyboard-Navigation (↑↓, Enter, Escape)
- Echtzeit-Suche ohne Latenz

Suchlogik:

```javascript
```

```
function calculateRelevanceScore(link, query) {
  let score = 0;
  const titleLower = link.title.toLowerCase();

  if (titleLower === query) score += 100;        // Exakte Übereinstimmung
  else if (titleLower.startsWith(query)) score += 50;  // Präfix-Match
  else if (titleLower.includes(query)) score += 25;    // Substring-Match

  // Weitere Scoring-Logik...
  return score;
}
```

4. Drag & Drop

Implementierte Features:

- Kategorie-Sortierung (horizontal/vertikal)
- Link-Repositionierung innerhalb Kategorie
- Link-Transfer zwischen Kategorien
- Touch-Support für mobile Geräte
- Auto-Scroll bei Drag-Operationen

Event-Handler:

```
javascript

// Kategorie Drag & Drop
handleCategoryDragStart(e)
handleCategoryDragOver(e)
handleCategoryDrop(e)

// Link Drag & Drop
handleLinkDragStart(e)
handleLinkDragOver(e)
handleLinkDrop(e)
```

5. Import/Export

Unterstützte Formate:

- Vollständige Konfiguration (JSON)
- Einzelne Kategorien (JSON)
- Einzelne Links (JSON)
- Default-Config (JavaScript)

Validierung:

- Schema-Validierung für alle Import-Formate
- Auto-Repair für fehlende Eigenschaften
- Konflikt-Resolution bei doppelten Namen
- Backward-Compatibility für ältere Versionen

API-Referenz

Globale Funktionen

Konfiguration

```javascript
loadConfig(): boolean
saveConfig(silent?: boolean): void
validateConfig(config: any): boolean
resetToDefault(): void
```

Kategorien

```javascript
toggleCategoryCollapsed(categoryKey: string): void
expandAllCategories(): void
collapseAllCategories(): void
editCategory(): void
deleteCategory(): void
```

Links

```javascript
editLink(): void
deleteLink(): void
toggleLinkFavorite(): void
openModal(): void
closeModal(): void
```

Suche

```javascript
```

```
performSearch(query: string): void
openSearchModal(): void
closeSearchModal(): void
navigateSearchResults(direction: number): void
```

Import/Export

javascript

```
exportConfig(): void
importConfig(event: Event): void
exportSingleLink(): void
importSingleLink(event: Event): void
exportSingleCategory(): void
importSingleCategory(event: Event): void
```

Event-System

Custom Events:

- `config-loaded`: Wird ausgelöst wenn Konfiguration geladen wurde
- `favorites-updated`: Bei Änderungen an Favoriten
- `search-performed`: Bei Suchvorgängen
- `drag-operation-complete`: Nach Drag & Drop-Operationen

Globale Variablen

javascript

```javascript
// Konfiguration
let config = {};
let linkCategories = {};
let categoryOrder = [];

// UI-State
let currentContextLink = null;
let isEditMode = false;
let favoritesOpen = false;

// Drag & Drop
let draggedElement = null;
let isDragging = false;

// Suche
let searchResults = [];
let selectedSearchIndex = -1;
```

Konfiguration

Standard-Konfiguration

Datei: `Config/default-config.js`

```javascript
const DEFAULT_CONFIG = {
  categories: {
    'Websites': {
      title: 'Websites',
      icon: '🌐',
      collapsed: false,
      links: [...]
    }
  },
  categoryOrder: ['Websites', 'Entwicklung', ...]
};
```

Anpassbare Parameter

CSS-Variablen (in styles.css):

```css
```

```
:root {
  --primary-color: #40e0d0;
  --secondary-color: #1a1a2e;
  --accent-color: #64ffda;
  --background-gradient: linear-gradient(135deg, #0f0f23 0%, #1a1a2e 50%, #16213e 100%);
}
```

JavaScript-Konstanten:

```
javascript

// Animation-Timing
const ANIMATION_DURATION = 300;
const TOAST_DURATION = 3000;

// Layout-Thresholds
const LARGE_CATEGORY_THRESHOLD = 4;
const GRID_BREAKPOINT = 350;
```

Emoji-Konfiguration

Kategorien erweitern:

```
javascript

EMOJI_DATABASE['custom_category'] = {
  name: 'Custom Category',
  icon: '🎯',
  emojis: [
    { emoji: '🚀', name: 'Rocket' },
    // ...
  ]
};
```

Deployment

Einfaches Deployment

1. **Dateien kopieren**: Alle Dateien auf Webserver kopieren
2. **Webserver konfigurieren**: Statische Dateien servieren
3. **HTTPS aktivieren**: Für localStorage-Zugriff empfohlen

Docker-Deployment

```
dockerfile
```

```
FROM nginx:alpine
COPY . /usr/share/nginx/html
EXPOSE 80
```

Apache .htaccess

apache

```
# Caching für statische Assets
<filesMatch ".(css|js|ico)$">
  ExpiresActive On
  ExpiresDefault "access plus 1 month"
</filesMatch>

# Kompression aktivieren
<IfModule mod_deflate.c>
  AddOutputFilterByType DEFLATE text/css text/javascript application/javascript
</IfModule>
```

Nginx-Konfiguration

nginx

```
server {
    listen 80;
    root /var/www/linksammlung;
    index Linksammlung.html;

    location ~* \.(css|js|ico)$ {
        expires 1M;
        add_header Cache-Control "public, immutable";
    }

    gzip on;
    gzip_types text/css application/javascript;
}
```

Performance

Optimierungen

CSS:

- CSS Grid für responsive Layouts
- Hardware-beschleunigte Animationen (transform, opacity)

- Kritisches CSS inline
- Lazy-Loading für große Emoji-Sets

JavaScript:

- Event-Delegation für dynamische Elemente
- Debounced Search-Input
- Virtual Scrolling für große Listen (geplant)
- Memory-Management für DOM-Nodes

Storage:

- Komprimierte JSON-Serialisierung
- Batch-Updates für localStorage
- Lazy-Loading von Konfigurationsteilen

Performance-Metriken

Zielwerte:

- First Paint: < 100ms
- Time to Interactive: < 500ms
- localStorage-Operations: < 10ms
- Drag & Drop-Latenz: < 16ms (60fps)

Monitoring

javascript

```javascript
// Performance-Tracking
const perfStart = performance.now();
// Operation...
const perfEnd = performance.now();
console.log(`Operation took ${perfEnd - perfStart} ms`);

// Memory-Monitoring
console.log(`Memory usage: ${JSON.stringify(performance.memory)}`);
```

Sicherheit

Content Security Policy

html

```
<meta http-equiv="Content-Security-Policy" content="
  default-src 'self';
  script-src 'self' 'unsafe-inline';
  style-src 'self' 'unsafe-inline';
  img-src 'self' data:;
  font-src 'self';
">
```

Input-Validierung

URL-Validierung:

```javascript
function validateUrl(url) {
  try {
    const urlObj = new URL(url);
    return ['http:', 'https:'].includes(urlObj.protocol);
  } catch {
    return false;
  }
}
```

XSS-Schutz:

```javascript
function sanitizeHtml(input) {
  const div = document.createElement('div');
  div.textContent = input;
  return div.innerHTML;
}
```

localStorage-Sicherheit

- Keine sensiblen Daten speichern
- JSON-Schema-Validierung bei jedem Load
- Backup-Mechanismus bei korrupten Daten
- Größenbegrenzung überwachen

Browser-Kompatibilität

Feature-Detection

```javascript
```

```javascript
// localStorage-Support
if (typeof Storage !== "undefined") {
  // localStorage verfügbar
} else {
  // Fallback oder Fehlermeldung
}

// Drag & Drop-Support
if ('draggable' in document.createElement('div')) {
  // Drag & Drop verfügbar
}

// CSS Grid-Support
if (CSS.supports('display', 'grid')) {
  // CSS Grid verfügbar
}
```

Polyfills

Für ältere Browser:

- `IntersectionObserver` für Lazy Loading
- `CustomEvent` für Event-System
- `Object.assign` für Object-Merging

Graceful Degradation

- Drag & Drop → Buttons für Sortierung
- CSS Animations → Statische Darstellung
- localStorage → Session-only Mode

Wartung

Debugging

Entwickler-Tools aktivieren:

```javascript
// Debug-Modus aktivieren
window.DEBUG_MODE = true;

// Debugging-Funktionen
window.getWelcomeLoaderDebug = () => window.extendedWelcomeLoader.getDebugInfo();
window.getConfigDebug = () => ({ config, linkCategories, categoryOrder });
```

Logging:

```javascript
// Fehler-Logging
window.addEventListener('error', (e) => {
  console.error('Application Error:', e.error);
});

// Performance-Logging
console.log('✅ Standard-Konfiguration geladen');
```

Backup-Strategien

1. **Automatischer Export** (geplant):

```javascript
setInterval(() => {
  if (hasChanges()) {
    autoExportConfig();
  }
}, 30000); // Alle 30 Sekunden
```

2. **Versioning**:

```javascript
const configWithVersion = {
  version: '2.6',
  timestamp: Date.now(),
  ...config
};
```

Updates

Version-Migration:

```javascript
```

```javascript
function migrateConfig(oldConfig, fromVersion, toVersion) {
  if (fromVersion < '2.5') {
    // Favoriten-Support hinzufügen
    addFavoritesSupportToLinks(oldConfig);
  }

  if (fromVersion < '2.6') {
    // Neue Features...
  }

  return oldConfig;
}
```

Monitoring

Error-Reporting:

javascript

```javascript
// Globaler Error Handler
window.addEventListener('error', (event) => {
  const errorReport = {
    timestamp: new Date().toISOString(),
    message: event.message,
    filename: event.filename,
    lineno: event.lineno,
    colno: event.colno,
    stack: event.error?.stack,
    userAgent: navigator.userAgent,
    url: window.location.href
  };

  console.error('Application Error Report:', errorReport);

  // Optional: An Monitoring-Service senden
  // sendErrorReport(errorReport);
});

// localStorage Überwachung
function monitorLocalStorage() {
  const used = JSON.stringify(config).length;
  const quota = 5 * 1024 * 1024; // 5MB typisch
  const percentage = (used / quota * 100).toFixed(2);

  console.log(`localStorage usage: ${used} bytes (${percentage}%)`);

  if (percentage > 80) {
    showMessage('Warnung: localStorage fast voll', 'warning');
  }
}
```

Health-Checks:

javascript

```javascript
function performHealthCheck() {
    const checks = {
        localStorageAvailable: typeof Storage !== "undefined",
        configValid: validateConfig(config),
        categoriesCount: Object.keys(linkCategories).length,
        linksCount: getTotalLinksCount(),
        favoritesCount: getTotalFavoritesCount(),
        lastSaved: localStorage.getItem('linkConfig_timestamp')
    };

    console.log('Health Check:', checks);
    return checks;
}

// Automatische Health-Checks
setInterval(performHealthCheck, 300000); // Alle 5 Minuten
```

Troubleshooting

Häufige Probleme und Lösungen:

1. **localStorage korrupt:**

```javascript
function repairLocalStorage() {
    try {
        const backup = localStorage.getItem('linkConfig_backup');
        if (backup) {
            localStorage.setItem('linkConfig', backup);
            showMessage('Backup wiederhergestellt', 'success');
        } else {
            resetToDefault();
            showMessage('Auf Standard zurückgesetzt', 'warning');
        }
    } catch (error) {
        console.error('Repair failed:', error);
    }
}
```

2. **Drag & Drop funktioniert nicht:**

```javascript
```

```javascript
function diagnosticDragDrop() {
  const support = {
    draggable: 'draggable' in document.createElement('div'),
    dataTransfer: 'DataTransfer' in window,
    dragEvent: 'DragEvent' in window,
    touch: 'ontouchstart' in window
  };

  console.log('Drag & Drop Support:', support);
  return support;
}
```

3. **Performance-Probleme:**

```javascript
function optimizePerformance() {
  // DOM-Nodes bereinigen
  document.querySelectorAll('.temporary-element').forEach(el => el.remove());

  // Event-Listener bereinigen
  cleanupEventListeners();

  // Drag & Drop State zurücksetzen
  cleanupAllTemporaryElements();

  console.log('Performance-Optimierung durchgeführt');
}
```

Testing

Unit-Tests (Beispiel-Framework):

```javascript
```

```
// Einfaches Test-Framework
class TestSuite {
  constructor(name) {
    this.name = name;
    this.tests = [];
  }

  test(description, testFn) {
    try {
      testFn();
      console.log(`✅ ${description}`);
    } catch (error) {
      console.error(`❌ ${description}:`, error);
    }
  }

  assertEquals(actual, expected, message = '') {
    if (actual !== expected) {
      throw new Error(`${message} - Expected: ${expected}, Actual: ${actual}`);
    }
  }
}

// Beispiel-Tests
const configTests = new TestSuite('Config Tests');

configTests.test('Should load default config', () => {
  const loaded = loadConfig();
  configTests.assertEquals(typeof config.categories, 'object');
  configTests.assertEquals(Array.isArray(categoryOrder), true);
});

configTests.test('Should validate config structure', () => {
  const validConfig = {
    categories: {
      test: {
        title: 'Test',
        icon: '🧪',
        collapsed: false,
        links: []
      }
    },
    categoryOrder: ['test']
  };
```

```
    configTests.assertEquals(validateConfig(validConfig), true);
});
```

Integration-Tests:

javascript

```
// E2E-Test-Simulation
async function testUserWorkflow() {
  console.log('💉 Starting E2E test simulation...');

  // 1. App laden
  loadConfig();
  renderCategories();

  // 2. Neue Kategorie erstellen
  const testCategory = {
    title: 'Test Category',
    icon: '💉',
    collapsed: false,
    links: []
  };

  linkCategories['test_category'] = testCategory;
  categoryOrder.push('test_category');

  // 3. Link hinzufügen
  linkCategories['test_category'].links.push({
    title: 'Test Link',
    url: 'https://example.com',
    description: 'Test Description',
    icon: '🔗',
    favorite: false
  });

  // 4. Favorit setzen
  linkCategories['test_category'].links[0].favorite = true;

  // 5. Suche testen
  performSearch('test');

  // 6. Export testen
  const exported = JSON.stringify(config);

  // 7. Cleanup
  delete linkCategories['test_category'];
  categoryOrder = categoryOrder.filter(key => key !== 'test_category');

  console.log('✅ E2E test completed successfully');
}
```

Entwicklungstools

Debug-Utilities:

javascript

```
// Entwickler-Hilfsmittel
window.devTools = {
  // Konfiguration inspizieren
  inspectConfig: () => {
    console.table(Object.entries(linkCategories).map(([key, cat]) => ({
      key,
      title: cat.title,
      linksCount: cat.links.length,
      collapsed: cat.collapsed
    })));
  },

  // Performance messen
  measureRenderTime: () => {
    const start = performance.now();
    renderCategories();
    const end = performance.now();
    console.log(`Render time: ${(end - start).toFixed(2)}ms`);
  },

  // Memory usage
  getMemoryUsage: () => {
    if (performance.memory) {
      const memory = performance.memory;
      return {
        used: `${(memory.usedJSHeapSize / 1024 / 1024).toFixed(2)} MB`,
        total: `${(memory.totalJSHeapSize / 1024 / 1024).toFixed(2)} MB`,
        limit: `${(memory.jsHeapSizeLimit / 1024 / 1024).toFixed(2)} MB`
      };
    }
    return 'Memory API not available';
  },

  // localStorage analysieren
  analyzeStorage: () => {
    const data = localStorage.getItem('linkConfig');
    if (!data) return 'No config found';

    return {
      size: `${(data.length / 1024).toFixed(2)} KB`,
      categories: Object.keys(JSON.parse(data).categories).length,
      totalLinks: getTotalLinksCount(),
      favorites: getTotalFavoritesCount()
    };
  },
```

```
  // Test-Daten generieren
  generateTestData: (categoryCount = 5, linksPerCategory = 10) => {
    for (let i = 0; i < categoryCount; i++) {
      const categoryKey = `test_category_${i}`;
      linkCategories[categoryKey] = {
        title: `Test Category ${i + 1}`,
        icon: ['📁', '🔧', '🎯', '🚀', '💡'][i % 5],
        collapsed: false,
        links: []
      };

      for (let j = 0; j < linksPerCategory; j++) {
        linkCategories[categoryKey].links.push({
          title: `Test Link ${j + 1}`,
          url: `https://example${j}.com`,
          description: `Test description for link ${j + 1}`,
          icon: '🔗',
          favorite: Math.random() > 0.8 // 20% Wahrscheinlichkeit für Favorit
        });
      }

      categoryOrder.push(categoryKey);
    }

    saveConfig(true);
    renderCategories();
    updateFavoritesCount();
    console.log(`Generated ${categoryCount} categories with ${linksPerCategory} links each`);
  },

  // Cleanup Test-Daten
  cleanupTestData: () => {
    const testKeys = Object.keys(linkCategories).filter(key => key.startsWith('test_'));
    testKeys.forEach(key => {
      delete linkCategories[key];
    });
    categoryOrder = categoryOrder.filter(key => !key.startsWith('test_'));

    saveConfig(true);
    renderCategories();
    updateFavoritesCount();
    console.log(`Cleaned up ${testKeys.length} test categories`);
  }
};

// Debug-Modus aktivieren
if (window.location.hostname === 'localhost' || window.location.search.includes('debug=true')) {
```

```
    window.DEBUG_MODE = true;
    console.log('🔧 Debug mode enabled. Use window.devTools for utilities.');
}
```

Erweiterungsmöglichkeiten

Plugin-System (Konzept)

javascript

```
// Plugin-Interface
class LinksammlungPlugin {
  constructor(name, version) {
    this.name = name;
    this.version = version;
    this.enabled = false;
  }

  onLoad() {
    // Plugin-Initialisierung
  }

  onCategoryRender(categoryElement, categoryData) {
    // Hook für Kategorie-Rendering
  }

  onLinkRender(linkElement, linkData) {
    // Hook für Link-Rendering
  }

  onConfigSave(config) {
    // Hook vor Config-Speicherung
    return config;
  }
}

// Plugin-Manager
window.pluginManager = {
  plugins: new Map(),

  register(plugin) {
    this.plugins.set(plugin.name, plugin);
    plugin.onLoad?.();
  },

  emit(event, ...args) {
    this.plugins.forEach(plugin => {
      if (plugin.enabled && plugin[`on${event}`]) {
        plugin[`on${event}`](...args);
      }
    });
  }
};
```

API-Erweiterungen

Webhook-System:

javascript

```javascript
// Webhook für externe Integration
class WebhookManager {
  constructor() {
    this.webhooks = [];
  }

  addWebhook(url, events) {
    this.webhooks.push({ url, events });
  }

  async trigger(event, data) {
    const relevantWebhooks = this.webhooks.filter(w => w.events.includes(event));

    for (const webhook of relevantWebhooks) {
      try {
        await fetch(webhook.url, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ event, data, timestamp: Date.now() })
        });
      } catch (error) {
        console.error(`Webhook failed: ${webhook.url}`, error);
      }
    }
  }
}
```

Synchronisation-API:

javascript

```
// Cloud-Sync-Interface
class SyncManager {
  constructor(apiEndpoint, apiKey) {
    this.apiEndpoint = apiEndpoint;
    this.apiKey = apiKey;
  }

  async syncUp() {
    const config = { ...window.config, timestamp: Date.now() };

    const response = await fetch(`${this.apiEndpoint}/sync`, {
      method: 'PUT',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${this.apiKey}`
      },
      body: JSON.stringify(config)
    });

    return response.ok;
  }

  async syncDown() {
    const response = await fetch(`${this.apiEndpoint}/sync`, {
      headers: { 'Authorization': `Bearer ${this.apiKey}` }
    });

    if (response.ok) {
      const remoteConfig = await response.json();
      // Merge-Logik hier
      return remoteConfig;
    }

    return null;
  }
}
```

Zukünftige Features

Geplante Erweiterungen:

1. **Tags-System** - Verschlagwortung von Links
2. **Themes** - Anpassbare Farbschemas
3. **Backup-Scheduler** - Automatische Backups
4. **Statistics** - Nutzungsstatistiken

5. **Collaboration** - Geteilte Kategorien
6. **Mobile App** - PWA-Funktionalität
7. **Browser-Extension** - Integration in Browser
8. **API-Server** - Backend für Synchronisation

Versionierung

Semantic Versioning

Format: MAJOR.MINOR.PATCH

- **MAJOR**: Breaking changes, neue Architektur
- **MINOR**: Neue Features, backward-compatible
- **PATCH**: Bugfixes, kleine Verbesserungen

Aktuelle Version: 2.6.0

Changelog-Format

markdown

```
## [2.6.0] - 2025-01-XX

### Added
- Erweiterte Emoji-Datenbank mit 15 Kategorien
- Welcome-Dialog für neue Benutzer
- Default-Config-Export-Funktion

### Changed
- Verbesserte Drag & Drop-Performance
- Optimierte Suchfunktion

### Fixed
- Favoriten-Counter-Bug behoben
- Mobile Touch-Events verbessert

### Deprecated
- Alte priority-Eigenschaft (wird zu favorite migriert)

### Security
- Input-Validierung verschärft
```

Lizenz und Compliance

Code-Qualität

Standards:

- ES6+ JavaScript
- Semantic HTML5
- WCAG 2.1 AA Accessibility
- Progressive Enhancement
- Mobile-First Design

Code-Reviews:

```javascript
// Code-Quality-Checks
const qualityChecks = {
  // Funktionen sollten nicht zu lang sein
  maxFunctionLength: 50,

  // Maximale Verschachtelungstiefe
  maxNestingDepth: 4,

  // Globale Variablen minimieren
  globalVariables: ['config', 'linkCategories', 'categoryOrder'],

  // Event-Listener-Management
  eventListenerCleanup: true,

  // Performance-Budget
  maxBundleSize: '50KB',
  maxInitTime: '500ms'
};
```

Browser-Standards

Compliance:

- HTML5 Validator
- CSS3 Validator
- WCAG 2.1 Guidelines
- JavaScript Standard Style
- Progressive Web App Standards

Support und Community

Issue-Tracking

Bug-Report-Template:

markdown

```
**Environment:**
- Browser: [Chrome/Firefox/Safari/Edge]
- Version: [Browser version]
- OS: [Windows/macOS/Linux]
- App Version: [2.6.0]

**Steps to Reproduce:**
1.
2.
3.

**Expected Behavior:**

**Actual Behavior:**

**Console Errors:**

**Additional Context:**
```

Feature-Request-Template:

markdown

```
**Problem Description:**

**Proposed Solution:**

**Alternative Solutions:**

**Impact:**
- [ ] High
- [ ] Medium
- [ ] Low

**Technical Considerations:**
```

Entwicklung

Development Setup:

bash

```
# Lokaler Development Server
python -m http.server 8000
# oder
npx serve .

# Live-Reload (optional)
npx live-server
```

Git-Workflow:

```
bash

# Feature-Branch erstellen
git checkout -b feature/new-feature

# Entwicklung...
git add .
git commit -m "feat: add new feature"

# Pull Request erstellen
git push origin feature/new-feature
```

Ende der technischen Dokumentation

Diese Dokumentation wird kontinuierlich aktualisiert. Letzte Änderung: Version 2.6.0